

FIT2014 Theory of Computation

Lecture 1 Introduction

slides by Graham Farr

Updated by KokSheik Wong for MUM

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

Warning

This material has been reproduced and communicated to you by or on behalf of Monash University in accordance with s113P of the Copyright Act 1968 (the Act).

The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice.

Languages

Computation is done with strings of symbols, so ...

Alphabets

An **alphabet** is a finite set of symbols.

Its members are called **letters** or **characters**.

We often denote it by Σ .

Examples of alphabets:

- ▶ $\{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z\}$
- ▶ $\{a, b\}$
- ▶ $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- ▶ $\{0, 1\}$

Languages

Words

A **word** is a finite string of symbols.

A **word over alphabet** Σ is a finite string of symbols all taken from Σ .

The **empty word** is the word of length 0, denoted by ε (or sometimes Λ).

Languages

A **language over alphabet** Σ is a set of words over Σ .

Special languages:

- ▶ **empty language:** $\emptyset = \{ \}$
 - ▶ not to be confused with the language containing just the *empty word*: $\{\varepsilon\}$
- ▶ $\Sigma^k := \{ \text{all words over } \Sigma \text{ of length } k \}$
 - ▶ E.g.: $\{a, b\}^2 = \{aa, ab, ba, bb\}$
- ▶ **universal language:** $\Sigma^* := \{ \text{all finite words over } \Sigma \}$

Languages

Natural languages

- ▶ English, Australian, Woiwurrung, Chinese, Auslan, . . . , Hebrew, Indonesian, Kannada, Hokkien, Croatian, Romanian, Russian, Tamil, Klingon, Spanish, Turkish, Tagalog, Vietnamese, French, Hungarian, Swedish, Elvish, . . .

Programming languages

- ▶ Python, Java, Haskell, awk, C, MIPS Assembler, Smalltalk, Prolog, Simula67, Interprogram, Algol-60, Cobol, Fortran, . . . , Rust, Java, C++, MATLAB, Racket, R, Lisp, UwU, Go, . . .

Languages also appear in:

- ▶ mathematics, music, knitting, games, . . .

Assumptions and notation

Unless otherwise stated, we use alphabet $\Sigma = \{a, b\}$.

Repetition: $a^2 = aa$, $ab^3 = abbb$, $(ab)^3 = ababab$

If x is a word, then x^k is the string obtained by concatenating k copies of x together:

$$x^k = \underbrace{xxx \cdots \cdots xx}_{k \text{ copies of } x}$$

$$(baa)^0 = \dots?$$

Some simple languages

EVEN-EVEN $:=$ {all strings in which *each* of a,b occurs an even number of times}
 $=$ $\{\epsilon, aa, bb, aaaa, aabb, abab, abba, baab, \dots\}$

Remember, 0 is even!

DOUBLEWORD $:=$ $\{xx : x \in \Sigma^*\}$
 $=$ {all strings obtained by concatenating some string with itself}
 $=$ $\{\epsilon, aa, bb, aaaa, , abab, baba, bbbb, aaaaaa, aabaab, \dots\}$

Note, $\epsilon\epsilon = \epsilon$.

PALINDROMES $:=$ {all strings that are the same forwards and backwards}
 $=$ $\{\epsilon, a, b, aa, bb, aaa, aba, bab, bbb, aaaa, , abba, baab, \dots\}$

Definitions, Theorems, Proofs

Definition

- ▶ specifies the precise meaning of something.

Theorem

- ▶ a mathematical statement that has been proved to be true.
- ▶ has some close but “less significant” relatives: Proposition, Lemma

Proof

- ▶ A step-by-step argument that establishes, logically and with certainty, that something is true.
- ▶ Should be verifiable.

Examples of theorems and proofs

Theorem.

English has a palindrome.

Proof. 'rotator' is an English word and also a palindrome.



An *existential statement* ...

There exists a palindrome in English

...just requires one suitable example for a proof.

Most proofs are not this short ...

Examples of theorems and proofs

Theorem.

Every English word has a vowel or a 'y'.

Proof.

'aardvark' has a vowel.

'aardwolf' has a vowel.

'aasvogel' has a vowel.

...

...

'syzygy' has a 'y'.

...

'zygote' has a vowel.



Theorems and proofs

To prove a *universal* statement . . .

For every English word, it has a vowel or a 'y'

. . . you need to cover every possible case.

One way is to go through all possibilities, in turn, and check each one.

But the number of things to check may be huge, or infinite.

So usually we want to reason in a way that can apply to many different possibilities at once.

Another example theorem

Theorem.

DOUBLEWORD $\subseteq$ EVEN-EVEN

Non-proof:

The examples on the previous slide show that every member of DOUBLEWORD has an even number of as and an even number of bs.

So every member of DOUBLEWORD is also a member of EVEN-EVEN.

“Proof by example” is not a proof.

...except where the Theorem just asserts the existence of a specific example!

Definitions, Theorems, Proofs

Theorem.

DOUBLEWORD $\subseteq$ EVEN-EVEN

Proof. Let $w \in \text{DOUBLEWORD}$.

Then $w = xx$ for some word x .

So, $\# \text{ a's in } w = 2 \times (\# \text{ a's in } x)$, so it's even.

Also, $\# \text{ b's in } w = 2 \times (\# \text{ b's in } x)$, so it's even too.

So $w \in \text{EVEN-EVEN}$. $\square$

To prove a subset relationship, $A \subseteq B$:

- ▶ Prove that every element of A is also an element of B .
 - ▶ Start with a general member of A .
 - ▶ Work towards proving that it also belongs to B .
- ▶ Give names to things.
- ▶ Use the definitions of the sets.

Other topics

- ▶ Propositional logic
- ▶ Predicates, quantifiers
- ▶ Linux
- ▶ Regular languages, finite automata
- ▶ Grammars, context-free languages, pushdown automata
- ▶ Lexical Analysis
- ▶ Introduction to parsing
- ▶ Turing Machines
- ▶ Computability, decidability
- ▶ Computational complexity
- ▶ Classes P and NP
- ▶ NP-completeness

Reading

- ▶ Sipser, pp 13–14
 - ▶ strings and languages
- ▶ Sipser, §0.3, pp 17–20
 - ▶ definitions, theorems, proofs